MCT - 454L Mobile Robotics

Lab Manual 5: Introduction to Gazebo and RViz with ROS 2 using TurtleBot3

Objective

The objective of this lab session is to familiarize students with simulation tools Gazebo and RViz in ROS 2 using the TurtleBot3 platform. By the end of this session, students will be able to visualize, control, and navigate the TurtleBot3 robot in a simulated environment, while exploring various features of RViz for sensor data visualization.

Prerequisites

- Basic knowledge of Linux commands
- Familiarity with ROS 2 concepts (nodes, topics, services)
- ROS 2 Humble installed on the system
- TurtleBot3 packages installed

Package Installation

1. **Update system packages:**

```
sudo apt update
```

2. **Install TurtleBot3 and Gazebo packages:**

TurtleBot3 is a low-cost, small-sized mobile robot designed for research and education purposes. It comes in two models, `burger` and `waffle`, and is equipped with sensors like LiDAR and IMU, making it ideal for SLAM (Simultaneous Localization and Mapping) and navigation tasks.

```
sudo apt install ros-humble-turtlebot3* ros-humble-gazebo-ros-pkgs
```

Environment Setup

1. **Configure TurtleBot3 model:**

Open terminal and set the TurtleBot3 model to `burger` (or `waffle` if applicable):

```
export TURTLEBOT3_MODEL=burger
```

- Add this line to `~/.bashrc` to avoid re-exporting in each session.

2. **Source ROS 2 workspace:**

```
source /opt/ros/humble/setup.bash
```

Launch Gazebo Simulation

Gazebo is a powerful open-source robotics simulation tool that allows users to create realistic 3D environments. It provides physics simulation, sensor models, and interfaces with robotic frameworks like ROS 2. Gazebo enables testing and development of robotic algorithms without requiring physical hardware.

1. Open a terminal and launch Gazebo with TurtleBot3 in an empty world:

```
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```

Observe the running nodes and topics using rqt_graph every time a launch file is executed.

2. Wait for Gazebo to fully load the simulated world with the TurtleBot3 model.

Visualize with Rviz and Generate Map

RViz (Robot Visualization) is a 3D visualization tool for ROS2. It helps visualize sensor data, robot models, coordinate frames, and navigation paths. RViz provides an interactive platform for debugging and analyzing robot behavior.

1. Open a new terminal and source the ROS 2 workspace if needed.
2. Launch RViz with Cartographer for simultaneous visualization and mapping:

Cartographer is a ROS package developed by Google for performing SLAM (Simultaneous Localization and Mapping) using sensor data from LiDAR and IMU. It allows the robot to build a map of the environment while simultaneously localizing itself within that map. Cartographer integrates seamlessly with ROS2, providing real-time mapping and pose estimation capabilities.

```
ros2 launch turtlebot3_cartographer cartographer.launch.py
use_sim_time:=true (write these both commands in single line)
```

3. RViz window will open displaying:
    - Odometry data at /odom
    - Laser scan data from LiDAR sensor
    - Map being built (if SLAM is running)

Exploring RViz Features

1. **Add Visualization Plugins:**
    - In RViz window, click on the `Add` button.
    - Select the following plugins (if not already added) from the list:
        - LaserScan
        - TF
        - Odometry
        - Path
        - Map
2. **Configure Display Settings:**
    - Adjust the frame settings under `Global Options` to `odom` or `map`.
    - Set the `Fixed Frame` to `map` to align all data with the world frame.
    - Modify color and size properties to enhance visibility.

Teleoperation

1. Open a new terminal and source the ROS 2 workspace if needed.
2. Run the keyboard teleoperation node:

```
ros2 run turtlebot3_teleop teleop_keyboard
```

3. Use the following keys to control the robot:
    - `w`: Forward
    - `s`: Backward
    - `a`: Turn left
    - `d`: Turn right
    - `q`: Increase speed
    - `z`: Decrease speed
    - `s`: Stop Forcefully

Tasks for Students

1. Navigate the robot using teleoperation to reach a designated target point in Gazebo.
2. Observe the Lidar scan and odometry data using Rviz's plugin.
3. Explore `TF` plugin in Rviz to visualize coordinate frames. Observe: How are the frames related?
4. Enable and customize the `Odometry` display to view real-time robot motion.
5. Record the robot's motion using `ros2 bag` and visualize the recorded data in RViz. Use the following command to record all topics: `ros2 bag record -a`
6. Use `Map` plugin to monitor the SLAM-generated map using Cartographer. Save the generated map using the following command:
   ```
   cd
   mkdir maps
   ros2 run nav2_map_server map_saver_cli -f maps/my_map
   ```
7. Try to teleop the TurtleBot3 back to (0, 0, 0) coordinates.
8. Create a publisher that alternates between publishing forward velocity and zero velocity every 2 seconds to the `/cmd_vel` topic using a timer callback.
9. Identify the message type being published to the `/odom` topic and create a subscriber that subscribes to the `/odom` topic and prints out the received messages. Paste the code and its running output in your submission.

Conclusion

This session provides a step-by-step guide to simulate and visualize the TurtleBot3 robot using Gazebo and RViz with ROS2. Students gain hands-on experience with essential simulation tools and learn how to interpret sensor data for robot navigation and mapping. These skills form the foundation for developing and testing advanced robot navigation algorithms in ROS2.

Deliverables

Students are required to submit the following deliverables at the end of the lab session:

1. A short report detailing the steps followed to complete the tasks.
2. Screenshots of the RViz visualization with TF, Odometry, LaserScan, and Path plugins enabled.
3. Recorded `ros2 bag` file demonstrating the robot's motion.
4. Screenshots of `rqt_graph` showing running nodes and topics
5. Observations on discrepancies between expected and simulated motion.

6. Code and running output for the `cmd_vel` publisher task.
7. Code and running output for the `odom` subscriber task.
8. Conclusion summarizing the overall learning experience and challenges faced.